

Diseño arquitectónico: analizador léxico

Documento de especificación de software

Asignatura: Compiladores · Grupo: 5 · Equipo: 02 · Lenguaje: Python

Este documento describe la arquitectura, las decisiones de diseño y los modelos de datos del analizador léxico (Scanner) del equipo. Se escribe desde cero, sin generadores léxicos como FLEX o PLY, y el autómata finito determinista (DFA) se implementa a mano.

1. Estructura del paquete y módulos

El código se empaqueta en el espacio de nombres `unam.fi.compilers.g5.02`, como pide la actividad, y se separa en módulos para dividir la entrada y salida, el motor léxico y las estructuras de datos.

```
unam/fi/compilers/g5/02/
├── __init__.py
├── main.py          # orquestador principal
├── io/
│   ├── reader.py   # carga completa en memoria (read())
│   └── writer.py    # reportes (consola + archivo txt)
├── lexer/
│   ├── scanner.py  # ciclo principal y Panic-Mode Recovery
│   └── dfa.py       # transiciones (grafo de funciones)
├── symbol_table/
│   └── symtab.py    # diccionario estructurado
├── models/
│   └── token.py     # tupla inmutable de clasificación
```

Por qué esta arquitectura

Cada módulo tiene una sola responsabilidad. El módulo `io/` lee el archivo completo en memoria con `read()` y no usa doble búfer. Los archivos de prueba son muy pequeños, así que la lectura directa es más rápida y hace más simple manejar el puntero de lectura y el lookahead.

Salida

`writer.py` imprime en consola el formato mínimo que pide la evaluación y, al mismo tiempo, genera un archivo de texto. Ese archivo sirve para documentar la evidencia en GitHub y adjuntarla al reporte técnico en Overleaf.

2. Definición de tokens y autómata (DFA)

Se eligió un enfoque mínimo que se puede ampliar. Aunque el equipo tiene la gramática completa de Solidity, por ahora el autómata solo implementa las expresiones regulares que hacen falta para los ejemplos obligatorios, agrupadas en siete categorías:

Categoría	Expresión regular interna	Ejemplos
Keyword	<code>^(printf int)\$</code>	<code>printf, int</code>
Identifier	<code>^[a-zA-Z_][a-zA-Z0-9_]*\$</code>	<code>main, a</code>
Operator	<code>^(= \+ - * /)\$</code>	<code>=, +, *</code>
Constants	<code>^[0-9]+\$</code>	<code>10, 256</code>
Literals	<code>^".*" \$</code>	<code>"This is an example"</code>
Punctuators	<code>^(\(\) ; ,)\$</code>	<code>(,), ;, ,</code>
Special_Characters	<code>^(\\$ @ #)\$</code>	<code>\$, @</code> (se capturan para el modo pánico)

Por qué el DFA usa funciones

El módulo `dfa.py` usa el patrón State: cada estado es una función de Python (por ejemplo, `estado_leyendo_id()`) y no hay una matriz de transiciones. Así no queda atado a una tabla fija y es más fácil agregar reglas de Solidity más adelante sin reescribir una tabla.

3. Manejo de errores léxicos

El escáner usa recuperación en modo pánico (Panic-Mode Recovery). Si el DFA encuentra un carácter que no lleva a ningún estado de aceptación, el análisis no se detiene. El motor aísla el carácter ilegal, muestra un mensaje de error en consola con la línea y la columna, descarta el lexema mal formado y sigue desde el siguiente carácter. De esta forma se pueden encontrar todos los errores en una sola pasada.

Identificadores

Como el caso de prueba es `int a = 10;` (sin el carácter especial `$`), el DFA solo acepta identificadores estándar, como en C o Solidity: empiezan con letra o guion bajo. Cualquier otro carácter, como `$`, `@` o `#`, entra al modo pánico. Esto mantiene el escaneo determinista.

4. Tabla de símbolos e inserción temprana

La tabla de símbolos se llena durante la fase léxica con las operaciones `Insert()` y `Lookup()`. Cuando el lexer reconoce una cadena con letras y dígitos, la busca en el diccionario de `symtab.py` sin esperar al analizador sintáctico. Si está en el subdiccionario de palabras reservadas, se emite un `Keyword`; si no, se inserta como `Identifier`.

La tabla es un diccionario de objetos con estructura. El lexer solo necesita saber que se trata de un identificador, pero se dejan reservados los campos que usarán las fases semánticas y de generación de código.

Name	Type	Size	Dimension	Line of declaration	Line of usage	Address
count	int	2	0	TBD	TBD	0xFF...
a	TBD	TBD	TBD	2	TBD	TBD

Con los valores TBD (por determinar), la tabla queda lista para cuando el parser y el analizador semántico necesiten validar el programa o generar el código de tres direcciones (Three Address Code). Además, para llevar el lexer a Solidity solo habría que agregar sus más de 80 palabras clave al diccionario inicial de `symtab.py`.

5. Propuesta de CFG y left factoring

Como requisito teórico se propone una gramática libre de contexto (CFG) para las instrucciones `printf("...");` e `int a = 10;`, con factorización por la izquierda para que sirva a un parser predictivo LL(1).

CFG original (no determinista):

```
S → Statement
Statement → Keyword ( ArgList ) ; | Keyword Identifier Operator Expression ;
ArgList → Literals | Literals , Identifier
Expression → Constants | Identifier
```

Después del left factoring

En `ArgList` se saca el prefijo común `Literals` y en `Statement` el prefijo `Keyword`:

```
S → Statement
Statement → Keyword StatementTail
StatementTail → ( ArgList ) ; | Identifier Operator Expression ;
ArgList → Literals ArgTail
ArgTail → , Identifier | ε
Expression → Constants | Identifier
```

Con esta transformación, un parser predictivo puede elegir la producción correcta con los conjuntos FIRST() y el siguiente símbolo (lookahead), sin hacer backtracking.

6. Instrucciones para el desarrollador

Esta sección indica lo que debe seguir quien programe el lexer, para que el código respete la arquitectura y las restricciones de la actividad.

A. Qué usar

- Patrón State: el autómata va en `dfa.py`, con métodos o funciones que representen los estados (por ejemplo, `estado_inicial(char)` y `estado_entero(char)`).
- Métodos nativos de cadenas de Python, como `char.isalpha()`, `char.isdigit()` y `char.isspace()`, con bloques `if/elif` para decidir las transiciones carácter por carácter.
- Lectura en bloque: `open(path).read()` en `io/reader.py`, con un puntero (índice numérico) que avanza sobre la cadena.
- Data classes o NamedTuples para definir el objeto Token, que debe ser inmutable, en `models/token.py`.

Sobre las expresiones regulares: las de la sección 2 son solo una descripción matemática de la gramática. No se debe usar el módulo `re` de Python para buscar tokens (por ejemplo, `re.findall()`). Un escáner hecho a mano recorre la cadena y cambia de estado con cada carácter que lee.

B. Qué no usar

- Generadores léxicos: no se puede instalar ni usar FLEX, Lex, PLY, SLY ni otra librería que genere la máquina de estados.
- Variables globales sueltas: el estado del escáner (línea, columna y puntero de lectura) debe estar dentro de la clase `Scanner`.
- Bucles sin salida: hay que detectar el fin de archivo (EOF) para terminar el ciclo de lectura.

C. Ruta de desarrollo del DFA

Para construir el DFA, el desarrollador debe seguir esta lógica:

1. Ciclo principal. En `scanner.py`, un bucle `while pointer < len(source_code)` que avanza carácter por carácter sobre el búfer de entrada.
2. Transición de estados (State Pattern). Cada vuelta empieza en `estado_inicial()` con el carácter actual:
 - Si `char.isalpha()` o `char == "_"`: pasa a `estado_leyendo_identificador()`. Consume letras, dígitos y guiones bajos. Al llegar a un delimitador (espacio o puntuador) se detiene, extrae el lexema y usa `Lookup()` para emitir Keyword o Identifier.
 - Si `char.isdigit()`: pasa a `estado_leyendo_constante()`. Consume dígitos hasta un carácter no numérico y emite Constants.
 - Si `char == "'"`: pasa a `estado_leyendo_literal()`. Consume todo hasta la comilla de cierre y emite Literals.
 - Si es un operador (por ejemplo `=` o `+`): emite Operator. Si es puntuación (por ejemplo `;` o `()`): emite Punctuators.
 - Si es un símbolo ajeno (por ejemplo `$`): se clasifica como `Special_Characters`, se activa el modo pánico, se registra la línea y se descarta.

3. Control de punteros (lookahead). No hay que consumir un carácter válido al salir de un estado.
Ejemplo: al leer `int a=10;;`, el estado de constante lee 1 y 0, y el siguiente carácter es ;. El estado debe detenerse sin consumirlo, para que `estado_inicial()` lo tome en la siguiente vuelta como Punctuators.